

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Scuola di Scienze Matematiche, Fisiche e Naturali
Corso di Laurea in Informatica
A.A. 2024-2025

# RELAZIONE LABORATORIO SISTEMI OPERATIVI
# SISTEMA DI TICKETING - HELP DESK


LUCA LORENZI - 7138834
luca.lorenzi@edu.unifi.it

MATTEO PINZANI - 7110442
matteo.pinzani@edu.unifi.it

COSIMO SGAMBELLURI - 7136287
cosimo.sgambelluri@edu.unifi.it

# 1

# INTRODUZIONE

## 1.1 OBIETTIVO

L'obiettivo del progetto consiste nello sviluppare, tramite socket del dominio **AF_INET**, un'infrastruttura per la realizzazione, gestione e consultazione di un sistema di richieste di supporto (Sistema di ticketing - Help Desk). Per realizzare quest'applicazione sono stati impiegati i seguenti strumenti:

- Linguaggio **C** per l'effettiva creazione del progetto (file sorgente e header).
- Libreria **SQLite3** per l'implementazione del database locale, contenente gli utenti registrati e i ticket creati.
- Libreria **GTK 3.0** per l'interfaccia grafica lato client.
- Libreria **yyjson** per la lettura e la gestione di documenti JSON.
- Software **Glade** per la creazione della GUI tramite il toolkit GTK.

## 1.2 ESECUZIONE

Per eseguire correttamente il progetto su Ubuntu/Debian, è prima necessario installare le librerie utilizzate (GTK e SQLite) e poi successivamente compilarlo.

### 1.2.1 *Installazione librerie*

```
sudo apt-get update
sudo apt-get install libsqlite3-dev libgtk-3-dev
```

### 1.2.2 *Compilazione ed esecuzione*

Dopo aver installato, eseguire:

```
make        # compila il progetto
./server    # esegue il server
./client    # esegue il client
```

# 2

# COMUNICAZIONE TRA CLIENT E SERVER

## 2.1 PROTOCOLLO

Per consentire la comunicazione tra client e server, e il relativo scambio di messaggi, abbiamo implementato un apposito protocollo. La comunicazione avviene tramite socket utilizzando il dominio **AF_INET** e il protocollo **JSON-RPC**.
JSON-RPC (JavaScript Object Notation Remote Procedure Call) è un protocollo di comunicazione leggero che permette di invocare metodi remoti attraverso la rete utilizzando il formato JSON per la serializzazione delle richieste. Il protocollo si basa su un modello request-response dove il client invia una richiesta JSON contenente il nome del metodo da invocare e i relativi parametri, mentre il server elabora la richiesta ed restituisce una risposta JSON con il risultato o eventuali errori.

### 2.1.1 *Request-Response*

*Request*

Ogni richiesta inviata dal client al server contiene i seguenti campi obbligatori:

```
{"jsonrpc": "2.0", "method": "methodName", "params": [], "id": 1}
```

- `jsonrpc`: stringa che specifica la versione del protocollo (sempre "2.0")
- `method`: stringa che identifica il metodo remoto da invocare sul server
- `params`: parametri del metodo sotto forma di array o oggetto JSON
- `id`: identificatore univoco della richiesta per la correlazione request-response

*Response*

Il server risponde con un messaggio JSON che mantiene lo stesso `id` della richiesta corrispondente. La struttura varia in base all'esito dell'operazione:

IN CASO DI SUCCESSO
```
{"jsonrpc": "2.0", "result": "", "id": 1}
```

- `result`: risultato dell'operazione richiesta; può essere una stringa, un numero, un oggetto o un array.
- `id`: stesso identificatore della richiesta originale

IN CASO DI FALLIMENTO

```
{"jsonrpc": "2.0", "error": {"code": -32601, "message": "Method not found"}, "id": 1}
```

- `error`: oggetto contenente informazioni sull'errore
  - `code`: codice numerico dell'errore secondo lo standard JSON-RPC
  - `message`: descrizione testuale dell'errore
- `id`: identificatore della richiesta che ha causato l'errore

### 2.1.2 *Codici di Errore Standard*

JSON-RPC 2.0 definisce codici di errore standardizzati per garantire interoperabilità:

- `-32700`: Parse error (JSON malformato)
- `-32600`: Invalid Request (richiesta non valida).
- `-32601`: Method not found (metodo non esistente).
- `-32602`: Invalid params (parametri non validi).
- `-32603`: Internal error (errore interno del server).
- `-32000 a -32099`: Server error (errori specifici dell'applicazione).

### 2.1.3 *Codice di Errore Personalizzati*

Il protocollo dà la possibilità di definire codici di errore personalizzati. Noi abbiamo scelto i seguenti:

- `-32401`: Login fallito.
- `-32404`: Errore nel recupero dei ticket.
- `-32501`: Errore nell'aggiornamento dei ticket.
- `-32500`: Errore nella creazione del ticket.
- `-32601`: Errore nel recupero degli utenti.

# 3

# CLIENT

Il file `client.c` implementa la logica principale del client nel sistema di ticketing: integra sia la parte di connessione alla rete, sia la gestione della GUI tramite GTK.

## 3.1 INIZIALIZZAZIONE

All'avvio, il programma inizializza il framework GTK con la funzione `gtk_init`. Successivamente, viene caricato un file di interfaccia utente in formato `.glade` che descrive la struttura grafica dei vari widget, quali pulsanti, finestre, dialogs ecc.
Il collegamento tra gli eventi grafici, come ad esempio il click di pulsanti o la chiusura di finestre e le funzioni di callback è gestito dalla funzione `g_signal_connect`.

## 3.2 CONNESSIONE AL SERVER

Dopo aver impostato l'interfaccia grafica, il programma procede con la creazione di un socket, il quale viene configurato per connettersi all'indirizzo IP di loopback (127.0.0.1) sulla porta 1420. Il tentativo di connessione al server viene eseguito attraverso un loop. Una volta stabilita la connessione, viene avviato il `gtk_main()`: il corrispondente blocco di codice resta in esecuzione finché l'interfaccia è attiva, permettendo all'utente di interagire con l'applicazione. Al termine dell'esecuzione (ad esempio, quando l'utente chiude la finestra), viene chiuso il socket con il server e il programma termina.

## 3.3 FUNZIONALITÀ DEI CLIENT

Le funzionalità variano a seconda della tipologia di utente:

- **Utenti Standard**: ciascun utente può creare nuovi ticket e visualizzare i propri.
- **Agenti**: possono visualizzare e modificare lo stato e la priorità dei ticket a loro assegnati.
- **Super-Agenti**: hanno le stesse capacità degli agenti ma possono inoltre visualizzare tutti i ticket nel sistema e assegnarli ad agenti specifici.

Tutti devono autenticarsi prima di poter utilizzare il programma e una volta autenticati, il controllo delle operazioni permesse viene effettuato lato client.

# 4

# SERVER

Il file `server.c` implementa il server del sistema di ticketing, il quale gestisce le richieste dei client, attraverso la funzione `handleClient()`. Inoltre, ogni principale azione del server, viene stampata in console attraverso l'utilizzo delle funzioni `logMessage()` e `logError()`.

## 4.1 FUNZIONAMENTO DEL `main`

La funzione `main()` inizia stampando che il server è in avvio. Successivamente viene creato il database e le relative tabelle con `createDb()` e `createTables()` se non esiste. Viene inizializzato un socket su `127.0.0.1:1420` e rimane in ascolto di nuove connessioni. Per ogni nuovo client il server:

- stampa l'indirizzo IP del client (se disponibile);
- crea un processo figlio con `fork()` per gestire quella connessione in modo indipendente;
- chiama la funzione `handleClient()`.

## 4.2 `handleclient()`

Questa funzione gestisce tutte le interazioni tra server e uno specifico client. Attende la lettura di una richiesta JSON-RPC e successivamente, in base al metodo richiesto, esegue l'azione corrispondente:

- **Login** (method = **login**): riceve email e password, verifica le credenziali con il database e risponde con i dati dell'utente in caso di successo oppure ritorna un errore.
- **Creazione ticket** (method = **createTicket**): riceve un oggetto JSON che rappresenta un `Ticket` e tenta di salvarlo nel database. Ritorna un errore se vi sono problemi.
- **Recupero ticket** (method = **getTickets**): riceve l'email dell'utente e restituisce la lista dei ticket ad esso associati. Se si tratta di un super agente vengono ritornati tutti i ticket. Ritorna un errore se vi sono problemi.
- **Aggiornamento ticket** (method = **updateTicket**): riceve i campi di un ticket che sono modificabili (priorità, stato e agente assegnato) e aggiorna la versione nel database. Ritorna un errore se vi sono problemi.
- **Recupero utenti** (method = **getUsers**): riceve un ruolo e restituisce la lista degli utenti corrispondenti. Ritorna un errore se vi sono problemi.

# 5

# DATABASE

Il file `sqlitedb.c` gestisce l'accesso al database SQLite, centralizzando le operazioni relative agli utenti e ai ticket. Questo modulo offre delle funzioni per l'apertura e l'inizializzazione del database, la creazione delle tabelle e le operazioni di inserimento, recupero e aggiornamento dei dati.

## 5.1 FUNZIONALITÀ PRINCIPALI

- Gestione della connessione con il database e creazione delle tabelle utenti e ticket; sono fornite funzioni per l'inizializzazione del database qualora esso non esista di già.
- Esecuzione di query per operazioni di creazione, visualizzazione e aggiornamento di utenti e ticket.

## 5.2 TABELLE

Il database contiene due tabelle:

- **User** con i campi: nome, cognome, email (chiave primaria), password e ruolo.
- **Ticket** con i campi: ID (chiave primaria), titolo, descrizione, data creazione, priorità, stato, creatore e agente assegnato.

### 5.2.1 *Sottoinsieme dei record della tabella User (per testare il codice)*

| Nome | Cognome | Email | Password | Ruolo* |
|---|---|---|---|---|
| Alice | Rossi | alice@example.com | pass1 | 0 |
| Irene | Rosa | irene@example.com | agent4 | 1 |
| Luca | Neri | luca@example.com | agent5 | 2 |

* 0 = utente standard, 1 = agente, 2 = super-agente

# 6

# GUI

La GUI consente agli utenti di autenticarsi, visualizzare, creare, modificare e filtrare le richieste sui ticket. Essa separa il design dalla logica, utilizzando in modo combinato la libreria GTK 3.0 e il file di layout `main.glade`. Questo file descrive l'aspetto dell'interfaccia attraverso XML, definendo quali widget esistono, come sono disposti e assegnando a ciascun un ID univoco. Il file è stato creato e modificato con un editor visuale, `Glade`.

## 6.1 `gui.c`

Contiene il codice C che dà vita all'interfaccia. Utilizzando l'oggetto `GtkBuilder` per caricare il file `main.glade`, questo codice accede ai singoli widget e li collega alla logica applicativa. L'interazione è resa possibile da un sistema di *segnali* e *callback*, dove le azioni dell'utente (come il clic su un pulsante) attivano specifiche funzioni C che ne gestiscono le conseguenze.

## 6.2 FUNZIONAMENTO

L'intera applicazione è *event-driven* e il suo flusso operativo è il seguente:

1. **Autenticazione**: l'utente inserisce le proprie credenziali in una finestra di login e la logica contatta il server per la validazione.
2. **Visualizzazione finestra principale**: in caso di successo, viene presentata la finestra principale, che mostra la lista dei ticket relativi all'utente.
3. **Interazione**: l'utente può compiere diverse azioni come l'aggiornamento della lista, la ricerca, o l'apertura di finestre di dialogo dedicate per creare un nuovo ticket o per visualizzare/modificare un ticket esistente. Ad ogni azione dell'utente corrisponde un segnale GTK che viene gestito da una funzione di callback specifica.

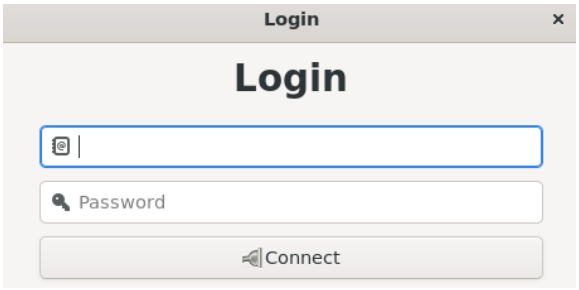
# 7

# ESEMPIO ESECUZIONE

Adesso mostriamo, tramite un esempio di esecuzione con schermate esplicative, il funzionamento dell'applicazione di ticketing: dall'accesso tramite login, alla gestione dei ticket (aggiunta, modifica e ricerca), fino alla disconnessione finale.

## 7.1 AVVIO E LOGIN

All'apertura dell'applicazione, l'utente si trova davanti alla finestra di login, in cui deve inserire le proprie credenziali (email e password). Questo permette l'autenticazione lato server, che verifica i relativi dati all'interno del database.
Nel caso in cui le credenziali non siano corrette, la GUI segnala che l'autenticazione non è andata a buon fine.

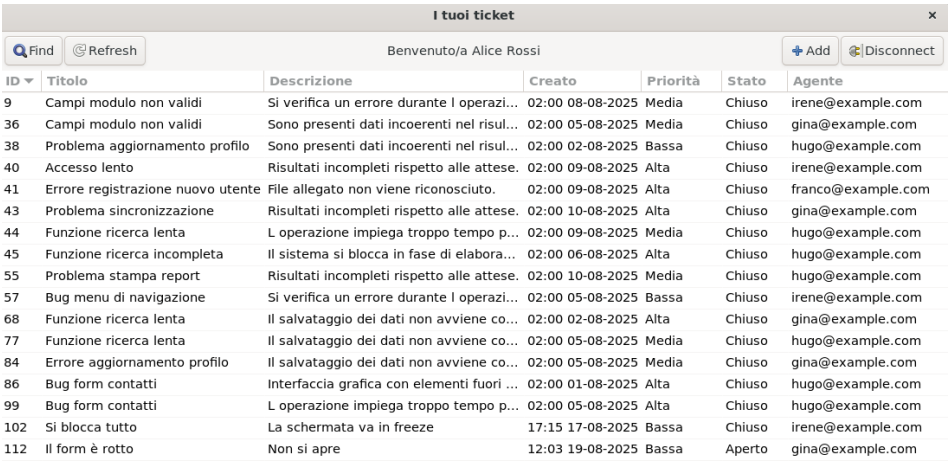






## 7.2 ACCESSO ALLA SCHERMATA PRINCIPALE

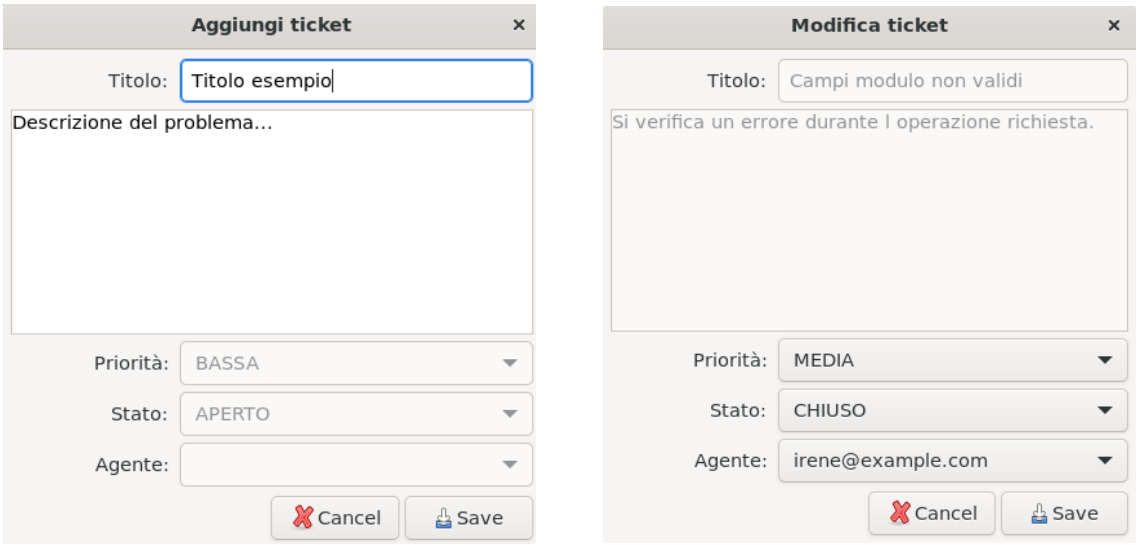
Dopo il login corretto, l'utente accede alla schermata principale, dove è visualizzato l'elenco dei ticket già presenti nel sistema, ossia lo storico delle sue richieste.




| ID | Titolo | Descrizione | Creato | Priorità | Stato | Agente |
|---|---|---|---|---|---|---|
| 9 | Campi modulo non validi | Si verifica un errore durante l operazi... | 02:00 08-08-2025 | Media | Chiuso | irene@example.com |
| 36 | Campi modulo non validi | Sono presenti dati incoerenti nel risul... | 02:00 05-08-2025 | Media | Chiuso | gina@example.com |
| 38 | Problema aggiornamento profilo | Sono presenti dati incoerenti nel risul... | 02:00 02-08-2025 | Bassa | Chiuso | hugo@example.com |
| 40 | Accesso lento | Risultati incompleti rispetto alle attese. | 02:00 09-08-2025 | Alta | Chiuso | irene@example.com |
| 41 | Errore registrazione nuovo utente | File allegato non viene riconosciuto. | 02:00 09-08-2025 | Alta | Chiuso | franco@example.com |
| 43 | Problema sincronizzazione | Risultati incompleti rispetto alle attese. | 02:00 10-08-2025 | Alta | Chiuso | gina@example.com |
| 44 | Funzione ricerca lenta | L operazione impiega troppo tempo p... | 02:00 09-08-2025 | Media | Chiuso | hugo@example.com |
| 45 | Funzione ricerca incompleta | Il sistema si blocca in fase di elabora... | 02:00 06-08-2025 | Alta | Chiuso | hugo@example.com |
| 55 | Problema stampa report | Risultati incompleti rispetto alle attese. | 02:00 10-08-2025 | Media | Chiuso | hugo@example.com |
| 57 | Bug menu di navigazione | Si verifica un errore durante l operazi... | 02:00 05-08-2025 | Bassa | Chiuso | irene@example.com |
| 68 | Funzione ricerca lenta | Il salvataggio dei dati non avviene co... | 02:00 02-08-2025 | Alta | Chiuso | gina@example.com |
| 77 | Funzione ricerca lenta | Il salvataggio dei dati non avviene co... | 02:00 05-08-2025 | Media | Chiuso | hugo@example.com |
| 84 | Errore aggiornamento profilo | Il salvataggio dei dati non avviene co... | 02:00 05-08-2025 | Media | Chiuso | gina@example.com |
| 86 | Bug form contatti | Interfaccia grafica con elementi fuori ... | 02:00 01-08-2025 | Alta | Chiuso | hugo@example.com |
| 99 | Bug form contatti | L operazione impiega troppo tempo p... | 02:00 05-08-2025 | Alta | Chiuso | hugo@example.com |
| 102 | Si blocca tutto | La schermata va in freeze | 17:15 17-08-2025 | Bassa | Chiuso | irene@example.com |
| 112 | Il form è rotto | Non si apre | 12:03 19-08-2025 | Bassa | Aperto | gina@example.com |

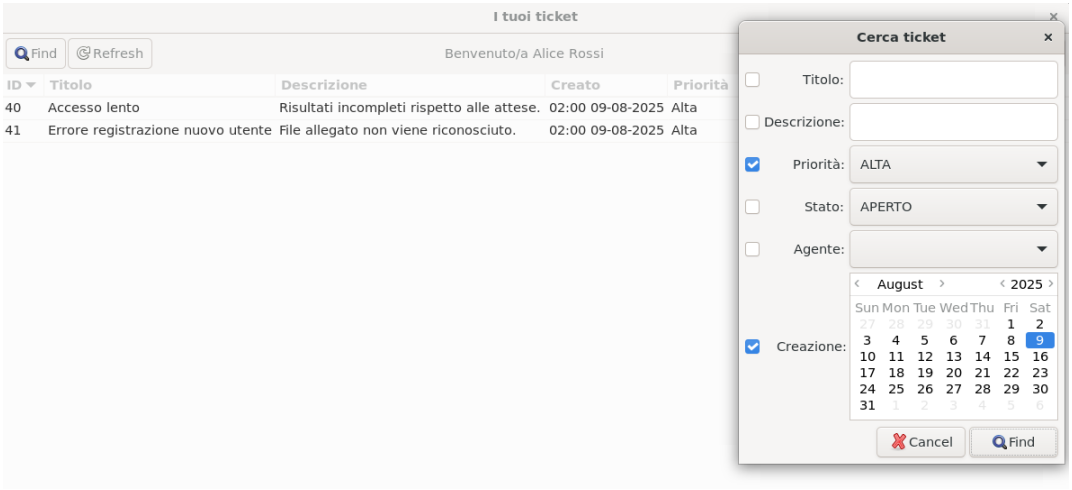
### 7.2.1 *Creazione e modifica di un ticket*

Se l'utente desidera aprire una nuova richiesta di assistenza può premere il pulsante **Add**: si aprirà una finestra, in cui inserire titolo e descrizione. Oltre alla creazione, l'applicazione permette anche di modificare un ticket già esistente; questa operazione può essere effettuata solo da un agente o un super-agente: l'agente deve cliccare due volte sul ticket e aggiornare i campi necessari. Il super-agente può anche modificare l'agente a cui il ticket è assegnato.




### 7.2.2 *Ricerca di un ticket*

Per cercare un ticket specifico, l'utente può utilizzare la funzione **Find**, che consente di filtrare i risultati tramite vari campi, quali titolo, descrizione, priorità, stato, agente e/o data di creazione.




### 7.2.3 *Aggiornamento della lista ticket*

Il pulsante **Refresh** permette di aggiornare l'elenco dei ticket, mostrando eventuali modifiche o nuove aperture effettuate da altri utenti.

### 7.2.4 *Disconnessione*

Infine, premendo **Disconnect**, l'utente viene disconnesso dal sistema: la sessione e l'applicazione vengono chiuse.

## 7.3 ESECUZIONE IN RETE LOCALE

Il software è stato anche testato su macchine diverse connesse alla stessa rete: è sufficiente modificare il file `server.c`, impostando il `sin_addr` non all'indirizzo di loopback ma ad un `inet_addr(... indirizzo IP del server ...)`. Anche lato client nel `s_addr` va inserito il corretto indirizzo del server al posto di quello di loopback sempre usando `inet_addr`.